LECTURE SERIES · DATABASE SYSTEMS

Database Normalization

From Anomalies to 1NF, 2NF, 3NF, BCNF and 4NF



Md. Rakib Trofder

Lecturer, Department of Computer Science & Engineering
Bangladesh Army University of Science & Technology

Anomalies · Functional Dependencies · 1NF → BCNF · Lossless Decomposition

THE PROBLEM WITH ONE BIG TABLE

Why Normalize?

Storing everything in one wide table repeats the same fact in many rows. Redundancy is not just wasted space — it is the source of three classic anomalies.

StuID	StuName	Dept	DeptHead	CourseID	CourseName	Instructor	Grade
S101	Rakib	CSE	Dr. Karim	CSE-3101	DBMS	Mr. Hasan	A
S101	Rakib	CSE	Dr. Karim	CSE-3103	OOP	Ms. Nadia	B+
S102	Sadia	CSE	Dr. Karim	CSE-3101	DBMS	Mr. Hasan	A-
S103	Tanvir	EEE	Dr. Rahim	EEE-2101	Circuits	Mr. Bari	B

Highlighted cells are the same fact stored again and again — that repetition is what breaks the table.

Insertion Anomaly

A new course with no enrolled student cannot be recorded — StuID is part of the key and cannot be left empty.

Update Anomaly

If Dr. Karim is replaced as CSE head, every CSE row must change. Miss one row and the database contradicts itself.

Deletion Anomaly

Delete the last row for EEE-2101 and the course, its instructor and the EEE department disappear with it.

THE FOUNDATION OF EVERY RULE

Functional Dependencies

X → Y reads “X determines Y”: any two rows that agree on X must agree on Y. Every normal form is simply a rule about which dependencies are allowed to survive.

FDs IN OUR EXAMPLE TABLE

`StuID → StuName, Dept`

a student's name and department

`Dept → DeptHead`

each department has one head

`CourseID → CourseName, Instructor`

each course has one title

`{StuID, CourseID} → Grade`

the full key determines grade

{StuID, CourseID} is the candidate key of the whole table.

Full Dependency

A non-key attribute depends on the entire composite key.

`{StuID, CourseID} → Grade`

Partial Dependency

A non-key attribute depends on only part of a composite key. Removed by 2NF.

`CourseID → CourseName`

Transitive Dependency

A non-key attribute depends on another non-key attribute. Removed by 3NF.

`StuID → Dept → DeptHead`

WHAT THE RULES ARE MEASURED AGAINST

Keys and Attribute Types

Normal forms are defined in terms of keys, so the vocabulary has to be exact before we start decomposing.

Super Key

Any attribute set that uniquely identifies a row — extra attributes allowed.

```
{StuID, CourseID, Grade}
```

Candidate Key

A minimal super key: remove any attribute and uniqueness is lost.

```
{StuID, CourseID}
```

Primary Key

The candidate key chosen by the designer to identify rows.

```
{StuID, CourseID}
```

Prime vs Non-Prime

Prime attributes belong to some candidate key; all others are non-prime.

```
Prime: StuID, CourseID
```

Rule of thumb: 2NF and 3NF talk about non-prime attributes; BCNF drops that exemption and talks about every determinant.

THE RULES, STEP BY STEP

The Ladder of Normal Forms

Each normal form adds one more rule. They are cumulative — a table must satisfy the previous form before it can reach the next one.

1NF — First Normal Form

Every attribute holds a single, atomic value — no repeating groups or multi-valued fields.

2NF — Second Normal Form

In 1NF and no partial dependency — non-key attributes depend on the whole key.

3NF — Third Normal Form

In 2NF and no transitive dependency — non-key attributes depend only on the key.

BCNF — Boyce–Codd NF

A stricter 3NF — every determinant must itself be a candidate key.

BEYOND BCNF

4NF removes independent multivalued dependencies; 5NF removes join dependencies. In practice, 3NF or BCNF is the design target.

STEP 0 · THE STARTING POINT

The Unnormalized Table (UNF)

Here is the raw enrolment record as a clerk might keep it in a spreadsheet. One row per student, with all of that student's courses packed into a single cell.

StuID	StuName	Dept	DeptHead	Courses (CourseID, CourseName, Instructor, Grade)
S101	Rakib	CSE	Dr. Karim	`CSE-3101, DBMS, Mr. Hasan, A ; CSE-3103, OOP, Ms. Nadia, B+`
S102	Sadia	CSE	Dr. Karim	`CSE-3101, DBMS, Mr. Hasan, A-`
S103	Tanvir	EEE	Dr. Rahim	`EEE-2101, Circuits, Mr. Bari, B`

Why this fails

- The Courses cell is not atomic — it holds four facts at once.
- There is no way to query “all students of CSE-3101” with plain SQL.
- The number of courses per student is unbounded, so no fixed set of columns can hold them.

THE PLAN FROM HERE

```
1NF  —  flatten the repeating group
2NF  —  remove partial dependencies
3NF  —  remove transitive dependencies
BCNF —  make every determinant a key
```

Each step splits a table; nothing is ever thrown away.

STEP 1 · ATOMIC VALUES ONLY

First Normal Form (1NF)

Rule: every cell contains exactly one indivisible value, there are no repeating groups, and every row is uniquely identifiable.

Fix: give each course its own row and repeat the student data. The key becomes the composite {StuID, CourseID}.

StuID	StuName	Dept	DeptHead	CourseID	CourseName	Instructor	Grade
S101	Rakib	CSE	Dr. Karim	CSE-3101	DBMS	Mr. Hasan	A
S101	Rakib	CSE	Dr. Karim	CSE-3103	OOP	Ms. Nadia	B+
S102	Sadia	CSE	Dr. Karim	CSE-3101	DBMS	Mr. Hasan	A-
S103	Tanvir	EEE	Dr. Rahim	EEE-2101	Circuits	Mr. Bari	B

Gold columns form the composite primary key.

What we gained

Every cell is queryable, joinable and indexable. The table is now a genuine relation.

What is still wrong

Student and course facts are duplicated on every enrolment row — partial dependencies remain.

STEP 2 · NO PARTIAL DEPENDENCY

Second Normal Form (2NF)

Rule: the table is in 1NF and every non-prime attribute depends on the whole composite key, not on a part of it.

DEPENDENCY AUDIT OF THE 1NF TABLE

`StuID → StuName, Dept, DeptHead` — partial

`CourseID → CourseName, Instructor` — partial

`{StuID, CourseID} → Grade` — full

Split the partial dependencies into their own tables — one per determinant.

STUDENT

StuID	StuName	Dept	DeptHead
S101	Rakib	CSE	Dr. Karim
S102	Sadia	CSE	Dr. Karim
S103	Tanvir	EEE	Dr. Rahim

COURSE

CourseID	CourseName	Instructor
CSE-3101	DBMS	Mr. Hasan
CSE-3103	OOP	Ms. Nadia
EEE-2101	Circuits	Mr. Bari

ENROLLMENT

StuID	CourseID	Grade
S101	CSE-3101	A
S101	CSE-3103	B+
S102	CSE-3101	A-
S103	EEE-2101	B

Course facts are now stored once, and a new course can be inserted before anyone enrols — the insertion anomaly is gone. STUDENT, however, still hides Dept → DeptHead.

STEP 3 · NO TRANSITIVE DEPENDENCY

Third Normal Form (3NF)

Rule: the table is in 2NF and no non-prime attribute depends on another non-prime attribute. Non-key attributes must depend on the key, the whole key, and nothing but the key.

BEFORE · STUDENT IN 2NF

StuID	StuName	Dept	DeptHead
S101	Rakib	CSE	Dr. Karim
S102	Sadia	CSE	Dr. Karim
S103	Tanvir	EEE	Dr. Rahim

`StuID → Dept → DeptHead`

DeptHead does not describe the student — it describes the department. The dependency reaches the key only through Dept, so it is transitive. Every CSE student repeats the head's name, and renaming the head means editing many rows.

AFTER · SPLIT ON THE DETERMINANT

STUDENT

StuID	StuName	Dept
S101	Rakib	CSE
S102	Sadia	CSE
S103	Tanvir	EEE

DEPARTMENT

Dept	DeptHead
CSE	Dr. Karim
EEE	Dr. Rahim

Now in 3NF

Dept in STUDENT is now a foreign key into DEPARTMENT. The head of a department is stored exactly once, a department can exist before any student joins it, and the original table is recovered by joining on Dept.

STEP 4 · EVERY DETERMINANT IS A KEY

Boyce–Codd Normal Form

3NF forgives a determinant that is a prime attribute. BCNF does not: for every dependency X → Y, X must be a candidate key. A fresh example shows the gap.

CLASS · IN 3NF, NOT IN BCNF

StuID	Course	Teacher
S101	DBMS	Hasan
S102	DBMS	Nadia
S101	OOP	Rafi
S103	DBMS	Hasan

{StuID, Course} → Teacher
Teacher → Course

Each course is taught by many teachers, but a teacher teaches only one course.

The violation

Teacher is a determinant, but Teacher alone is not a candidate key. 3NF allows it because Course is a prime attribute — BCNF does not.

The cost is real: the fact that Hasan teaches DBMS is repeated on every one of his enrolment rows, and it vanishes if his last student drops the course.

DECOMPOSE ON THE KEY

TEACHES

Teacher	Course
Hasan	DBMS
Nadia	DBMS
Rafi	OOP

STUDENT_TEACHER

StuID	Teacher
S101	Hasan
S102	Nadia
S101	Rafi
S103	Hasan

Trade-off: the split is lossless, but {StuID, Course} → Teacher can no longer be checked without a join — BCNF sometimes costs dependency preservation.

BEYOND BCNF · MULTIVALUED DEPENDENCIES

Fourth Normal Form (4NF)

When one key has two independent multivalued facts, a BCNF table still forces every combination to be stored. 4NF removes that cartesian product.

STUDENT_ACTIVITY · IN BCNF, NOT IN 4NF

StuID	Language	Sport
S101	Bangla	Cricket
S101	Bangla	Football
S101	English	Cricket
S101	English	Football
S102	Bangla	Chess

2 languages × 2 sports = 4 rows for one student, none of which is a real fact.

Five rows become six across two tables — but each row now states one independent fact, and the join restores the original.

The MVD

`StuID ↠ Language`
`StuID ↠ Sport`

A student's languages and sports are unrelated to each other. Adding one new sport forces a new row for every language already listed — pure redundancy with no functional dependency to blame.

SPLIT INTO TWO RELATIONS

STUDENT_LANGUAGE

StuID	Language
S101	Bangla
S101	English
S102	Bangla

STUDENT_SPORT

StuID	Sport
S101	Cricket
S101	Football
S102	Chess

THE RESULT OF THE WHOLE PROCESS

The Normalized Schema

One wide table has become four focused relations. Every fact is stored exactly once and every table has a clear reason to exist.

DEPARTMENT

```
Dept  (PK)
DeptHead
```

One row per department. The head's name lives here and nowhere else.

STUDENT

```
StuID  (PK)
StuName
Dept  (FK)
```

One row per student; Dept is a foreign key into DEPARTMENT.

COURSE

```
CourseID  (PK)
CourseName
Instructor
```

One row per course, independent of who enrols.

ENROLLMENT

```
StuID  (PK, FK)
CourseID  (PK, FK)
Grade
```

The link table — both columns are foreign keys, and the only place a grade appears.

READING THE SCHEMA BACK

```
SELECT s.StuName, d.DeptHead, c.CourseName, e.Grade FROM ENROLLMENT e JOIN STUDENT s USING (StuID) JOIN DEPARTMENT d USING (Dept) JOIN COURSE c USING (CourseID);
```

TESTING A DECOMPOSITION

Lossless Join & Dependency Preservation

Splitting a table is only correct if nothing is lost. Two properties decide whether a decomposition is safe.

Lossless Join

Joining the pieces must give back the original rows — no more and no fewer.

$R1 \cap R2 \rightarrow R1 \quad \text{or} \quad R1 \cap R2 \rightarrow R2$

The shared attributes must be a key of at least one piece. STUDENT ∩ DEPARTMENT = Dept, and Dept is the key of DEPARTMENT — so the split is lossless.

Dependency Preservation

Every original FD should still be checkable inside one table, without a join.

$F^{+} = (F1 \cup F2 \cup \ldots)^{+}$

Otherwise the DBMS must join tables on every insert to enforce a constraint — expensive, and easy to get wrong.

The practical rule

A lossless, dependency-preserving 3NF decomposition always exists. A lossless BCNF decomposition always exists too — but it may not preserve every dependency. When the two conflict, most designers stop at 3NF.

PUTTING IT TOGETHER

Normalizing a Table in Practice

A repeatable procedure you can apply to any messy table in an exam or a real project.

1 List the attributes

Write out every column the table holds, with a few sample rows.

2 Write the FDs

State every dependency the business rules imply, not just the obvious ones.

3 Find the candidate keys

Compute attribute closures; mark prime and non-prime attributes.

4 Apply 1NF then 2NF

Flatten repeating groups, then split off each partial dependency.

5 Apply 3NF then BCNF

Split off transitive dependencies, then any determinant that is not a key.

6 Verify the split

Check lossless join and dependency preservation before you commit.

AND WHEN NOT TO

Denormalization is a deliberate, measured exception — a read-heavy report may duplicate a column for speed. Normalize first; denormalize only with evidence.